

NeoGRC

Implementor Manual

Evidence-driven governance, risk and compliance for Frappe / ERPNext v15

Automated evidence collection · multi-framework crosswalk · gap assessment

Application	neogrc
Version	0.2.0
Target platform	Frappe / ERPNext v15
Licence	MIT
Document date	12 August 2026
Audience	Implementation consultants and site administrators

Contents

1 Introduction

- 1.1 What problem this solves
- 1.2 What it is not
- 1.3 Relationship to AlphaX GRC

2 Architecture

- 2.1 Separation of collection from interpretation
- 2.2 Data model
- 2.3 Engineering conventions

3 Installation

- 3.1 Prerequisites
- 3.2 Install
- 3.3 What install creates
- 3.4 First run without credentials
- 3.5 Uninstall

4 Roles and permissions

- 4.1 Segregation of duties on exceptions

5 Evidence collection

- 5.1 Connectors, fetchers and sets
- 5.2 Seeded connectors
- 5.3 The internal connector
- 5.4 Preflight
- 5.5 Exit codes
- 5.6 Declarative query fetchers
- 5.7 Validation rules
- 5.8 Shell fetcher sandboxing

6 The finding contract

- 6.1 Status and severity are independent
- 6.2 Contract rules enforced at ingest
- 6.3 Ingest API

7 Crosswalk and gap assessment

- 7.1 Two-hop resolution
- 7.2 Running an assessment
- 7.3 How controls are scored
- 7.4 Multi-framework optimisation

8 Operations

- 8.1 Scheduled jobs
- 8.2 Reports
- 8.3 Data residency
- 8.4 Troubleshooting

9 Licensing and attribution

- 9.1 Framework status
- 9.2 Loading licensed text
- 9.3 The build-time guard
- 9.4 Reference projects

10 Extending the application

- 10.1 Adding a connector
- 10.2 Adding a framework
- 10.3 Development checks
- 10.4 Suggested next steps

Figures: 1 architecture · 2 data model · 3 evidence run lifecycle · 4 crosswalk resolution · 5 gap assessment scoring

1 Introduction

NeoGRC is a Frappe v15 application that turns technical evidence into defensible compliance positions. Connectors collect evidence from systems that already exist in the estate. Validation rules decide whether that evidence demonstrates a control. Findings are recorded against a versioned data contract, expanded through a control crosswalk, and scored into a gap assessment you can put in front of an auditor.

The design assumption throughout is that compliance evidence has to survive being questioned. A screenshot in a shared folder does not. A hashed artifact, attached to a timestamped run, produced by a named fetcher, validated by a stated rule, mapped to a specific control, does.

1.1 What problem this solves

An organisation certifying against several frameworks at once — say NCA ECC-2 for the regulator, ISO 27001 for customers, and PCI DSS for the acquiring bank — will typically run three evidence collection exercises, produce three binders, and discover the same encryption control three times. The cost is not the certification; it is the triplication.

This application collapses that. A connector emits one finding against one canonical control. The crosswalk resolves it into every framework that control touches. One collection run answers all of them.

1.2 What it is not

- It is not a policy management system with an approval workflow bolted on. Policies exist here as records that evidence can attach to, not as a document management product.
- It is not an agent. Nothing is installed on monitored hosts. Collection is pull-based, from the site outward, using credentials the site owner provisions.
- It does not ship the normative text of any licensed standard. See section 9.
- It does not replace an auditor's judgement. It produces the evidence an auditor asks for, and shows honestly where evidence is absent.

1.3 Relationship to AlphaX GRC

AlphaX GRC covers governance and risk quantification — the programme layer. NeoGRC covers the engineering layer beneath it: automated evidence, connector findings, crosswalk, and gap assessment. The two are independent apps with no hard dependency, and either can be installed alone.

If both are installed and you prefer a single control register, make the AlphaX GRC control set the canonical framework in GRC Engine Settings. Nothing else in the application changes; the crosswalk simply pivots through a different hub.

2 Architecture

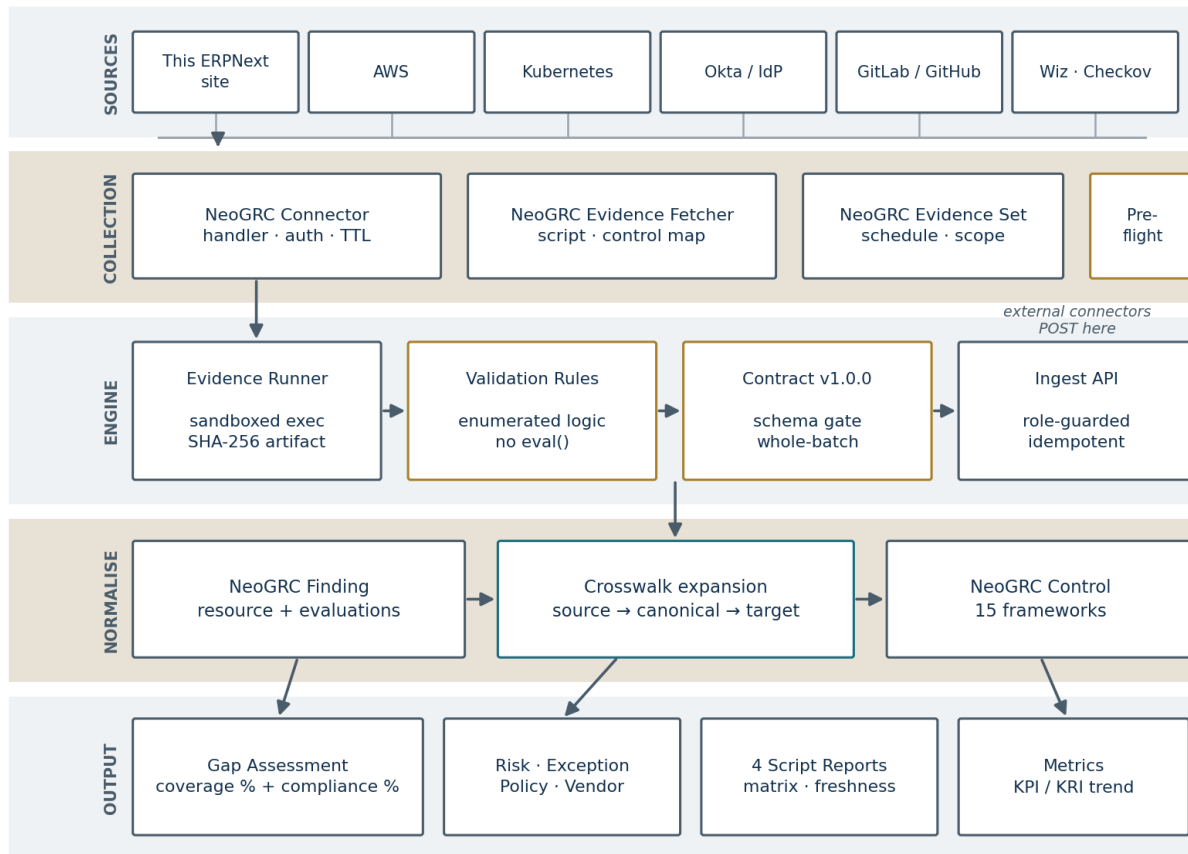


Figure 1 — End-to-end architecture, from source systems to assurance output

Five layers, each with a single responsibility.

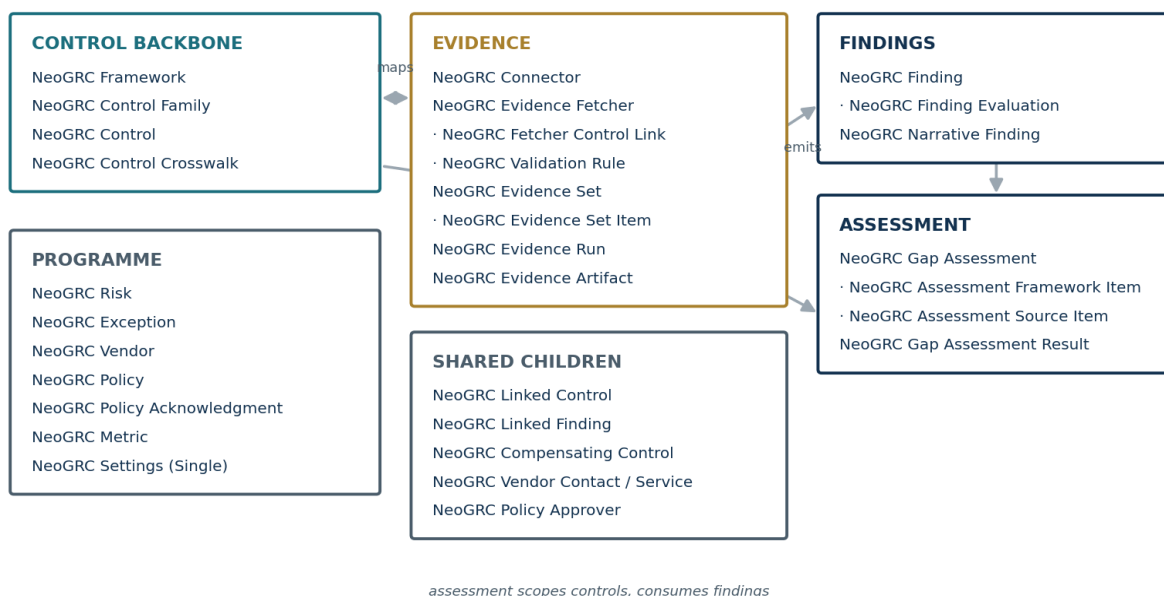
Layer	Responsibility	Key DocTypes
Sources	Systems that hold the truth. Not modified by this app.	—
Collection	Describes what to collect, from where, how often, and with what credentials.	NeoGRC Connector, GRC Evidence Fetcher, GRC Evidence Set
Engine	Executes collection, validates output, enforces the data contract.	NeoGRC Evidence Run, GRC Evidence Artifact
Normalisation	Converts raw results into control-scoped findings and expands them across frameworks.	NeoGRC Finding, GRC Control, GRC Control Crosswalk
Output	Scores the position and reports it.	NeoGRC Gap Assessment, GRC Metric, Script Reports

2.1 Separation of collection from interpretation

A connector never needs to know which frameworks the organisation reports against. It emits a result against a canonical control identifier and stops there. Framework resolution happens later, in the crosswalk, entirely inside this application.

This matters operationally more than it sounds. When the organisation adds PCI DSS to its obligations next year, no connector is touched and no evidence is re-collected. You add crosswalk edges. The existing evidence starts answering the new framework the same day.

2.2 Data model



32 DocTypes · child tables marked with a dot · ~5,000 lines of controller Python

Figure 2 — The 31 DocTypes, grouped by role. Child tables are marked with a leading dot.

Thirty-one DocTypes across five groups. The control backbone is master data seeded at install. The evidence group is configuration the implementor builds. Findings and assessment are transactional. The programme group holds the GRC records that give findings business context — risks, exceptions, vendors, policies and metrics.

2.3 Engineering conventions

Convention	Rationale
No Server Scripts. All logic in versioned Python controllers.	Server Scripts are site data, not code. They do not appear in a diff, do not survive a fresh install, and cannot be reviewed before deployment.
Patches declared under <code>[post_model_sync]</code> .	Every seeder writes to DocTypes that the same migrate creates. Under <code>[pre_model_sync]</code> they would run against tables that do not yet exist.

Convention	Rationale
Idempotent <code>after_install</code> and <code>after_migrate</code> .	Seeders create if missing and never overwrite. A customer who edits a seeded control keeps that edit through every subsequent upgrade.
Heavy operations are background jobs.	Evidence collection and gap assessment call external systems and scan large result sets. Neither may hold a web worker.
Every whitelisted endpoint is role-guarded.	A whitelisted method is a public HTTP endpoint. <code>verify_tree.py</code> fails the build if one lacks a permission check.

3 Installation

3.1 Prerequisites

Requirement	Detail
Frappe framework	v15.x. The app declares <code>frappe >=15, <16</code> .
Python	3.10 or later, as required by Frappe v15.
ERPNext	Not required. The app is standalone, though the <code>erpnext-internal</code> connector collects richer evidence when ERPNext is present.
Background workers	Must be running. Evidence collection and gap assessment are queued jobs.
Scheduler	Must be enabled for scheduled evidence sets and daily hygiene jobs.

3.2 Install

```
cd ~/frappe-bench
bench get-app neogrc /path/to/neogrc
bench --site yoursite install-app neogrc
bench --site yoursite migrate
```

Confirm the scheduler is live, otherwise scheduled evidence sets will never fire:

```
bench --site yoursite doctor
bench --site yoursite enable-scheduler
```

3.3 What install creates

Object	Count	Note
Frameworks	15	One canonical (NCC), fourteen reference
Control families	16	Governance through Secure Development
Canonical controls	62	Original wording, freely editable
Framework controls	640	Real identifiers — see section 9
Crosswalk edges	1,121	211 binding, 910 family-level derivations
Connectors	9	Two enabled, seven disabled
Fetchers	38	11 internal Python, 9 declarative query, 18 shell
Evidence sets	5	Three enabled, two disabled
Roles	3	NeoGRC Manager, NeoGRC Engineer, NeoGRC Auditor

Every seeder is idempotent

Seeding runs on both `after_install` and `after_migrate`, creating only what is missing. Editing a seeded control, disabling a seeded fetcher, or rewriting a stub framework's titles are all safe — a later bench migrate will not undo them.

3.4 First run without credentials

The `ernext-internal` connector ships enabled and collects evidence from the site itself. It needs no credentials and makes no outbound connection, so the application is demonstrable on install day.

1. Open GRC Evidence Set and select ES-ERP-BASELINE.
2. Click Run Now. The set is queued to the background worker.
3. Open the resulting GRC Evidence Run. Each fetcher has an artifact with a SHA-256 hash and a recorded validation outcome.
4. Open GRC Finding to see the emitted findings, each scoped to a canonical control.
5. Create a GRC Gap Assessment, add framework NCC in the scope table, and click Run Assessment.
6. Read the scorecard: automated coverage and compliance, reported separately.

Expect a low coverage figure on this first pass. Eleven internal fetchers cannot evidence sixty-two controls, and the assessment says so rather than concealing it. That is the intended behaviour, and section 7.3 explains why.

3.5 Uninstall

`before_uninstall` raises a warning listing the evidence artifacts, findings and submitted assessments that will be destroyed. Submitted gap assessments are audit records; export them before removing the application.

4 Roles and permissions

Role	Intended holder	Permissions
NeoGRC Manager	Compliance owner, CISO delegate	Full control including delete on audit records, and approval of exceptions
NeoGRC Engineer	Security engineer, implementation consultant	Create and edit connectors, fetchers and evidence sets; run collection and assessments; cannot delete audit records
NeoGRC Auditor	Internal audit, external assessor, regulator liaison	Read-only across controls, findings, evidence, artifacts and assessments

The auditor role exists so an external assessor can be given a site login rather than an export. They see the evidence in place, with its hashes and run history intact, and can change nothing.

4.1 Segregation of duties on exceptions

An exception is a documented decision not to meet a control. The controller blocks a user from approving an exception they raised themselves, regardless of role. A NeoGRC Manager who creates an exception must have a second NeoGRC Manager approve it.

This is deliberately not configurable. An exception register where the requester is the approver documents nothing, and it is the first thing a competent auditor tests.

4.2 Policy acknowledgment

Approving a policy issues one NeoGRC Policy Acknowledgment row per active employee, or per enabled desk user on sites without ERPNext. Rows are keyed to the policy version, so reissuing a policy at v3 starts a fresh round of sign-off rather than inheriting v2's.

Only the subject may mark their own row acknowledged. A manager can create, reassign and delete rows but cannot sign for someone else, because an acknowledgment register signed off in a bulk edit is evidence of nothing. Awareness coverage then becomes measurable rather than asserted, and the EVD-Q-EMPLOYEE-ACK query fetcher reports it as a control.

5 Evidence collection

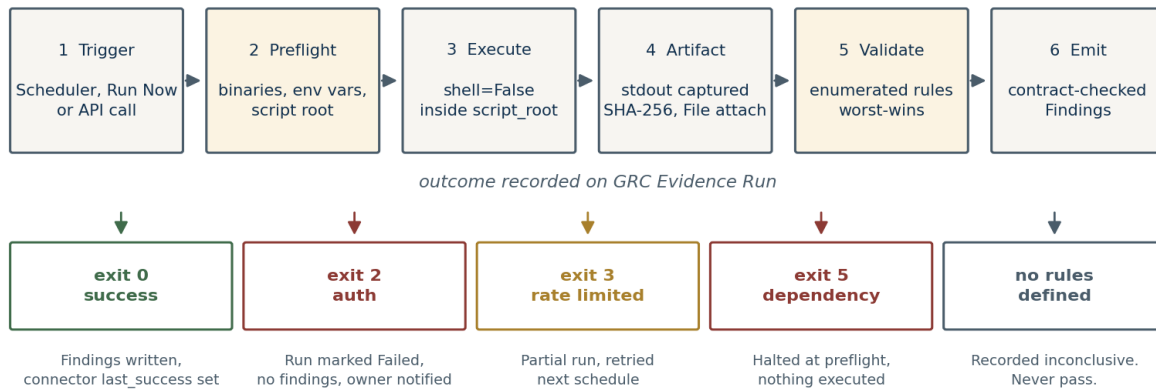


Figure 3 — The evidence run lifecycle and its terminal states

5.1 Connectors, fetchers and sets

Object	Describes	Example
NeoGRC Connector	A source system: handler type, auth mode, required binaries and environment variables, cache TTL.	aws-inspector — shell handler, environment-variable auth, 24 hour TTL
NeoGRC Evidence Fetcher	One check within a connector, carrying its control mappings and validation rules.	EVD-BLOCK-ENC — are all EBS volumes encrypted?
NeoGRC Evidence Set	A named group of fetchers with a schedule and a target framework.	ES-KSA-ECC-CORE — monthly, scoped to NCA ECC-2

5.2 Seeded connectors

Connector	Type	Fetchers	Ships
erpNext-internal	Python	11	Enabled
erpNext-query	Internal Query	9	Enabled
aws-inspector	Shell	11	Disabled
k8s-inspector	Shell	2	Disabled
okta-inspector	Shell	2	Disabled
gitlab-inspector	Shell	2	Disabled
github-inspector	Shell	—	Disabled

Connector	Type	Fetchers	Ships
wiz-inspector	Shell	—	Disabled
checkov-inspector	Shell	1	Disabled

External connectors ship disabled, with no script root

Enabling a connector implies provisioning credentials and permitting outbound network egress from the site. That is a decision for the site owner, made deliberately, not a default inherited from an installer.

5.3 The internal connector

Eleven Python handlers collect from the site itself. They are the reason the application is useful before any cloud onboarding work begins.

Fetcher	Collects
EVD-ERP-2FA	Two-factor authentication enforcement and per-user enrolment
EVD-ERP-PRIV	Users holding System Manager and other privileged roles
EVD-ERP-LEAVER	Disabled users still holding role assignments
EVD-ERP-PWPOLICY	Password policy strength settings in force
EVD-ERP-AUDITLOG	Audit logging and document versioning configuration
EVD-ERP-FAILEDLOGIN	Failed login volume and lockout thresholds
EVD-ERP-BACKUP	Most recent backup and its age
EVD-ERP-APPS	Installed app inventory with versions
EVD-ERP-EXCEPTIONS	Exceptions past expiry still marked active
EVD-ERP-POLICYREVIEW	Policies past their review date
EVD-ERP-VENDORREV	Vendors past their tier-driven review cadence

The final three are worth noting: the application collects evidence about its own hygiene. An expired exception still treated as live, or a policy two years past review, is a finding like any other.

5.4 Preflight

Before the first fetcher in a set executes, preflight verifies the connector's declared binaries are on PATH, its required environment variables are set, and its script root exists and is readable. A missing AWS CLI produces one clear exit-code-5 message instead of thirty opaque failures thirty seconds apart.

5.5 Exit codes

Code	Meaning	Engine behaviour
0	Success	Artifact recorded, rules evaluated, findings emitted, connector last-success timestamp updated
1	Generic failure	Run marked Failed, stderr captured on the run record
2	Authentication failure	Run marked Failed, no findings emitted, connector owner notified
3	Rate limited	Run marked Partial, retried at the next scheduled interval
4	Partial result	Findings emitted for what was collected, run flagged Partial
5	Dependency missing	Halted at preflight, nothing executed

A failed run never emits findings. This is not a detail — silently emitting a pass because a collection script died before it could find anything is precisely the failure mode that makes automated compliance tooling untrustworthy.

5.6 Declarative query fetchers

A fetcher can run a query instead of a script. Set Handler Mode to Query and give it a source DocType, a Frappe filter object, and a pass condition. Nothing is deployed and no Server Script is created.

Condition	Passes when
COUNT_EQUALS_ZERO	Nothing matched the filters — the usual no-exceptions check
COUNT_LTE_THRESHOLD	Match count is at or below the threshold
COUNT_GTE_THRESHOLD	At least N records exist, for example approved policies
ALL_ROWS_FIELD_SET	Every matched row has a value in the checked field
ALL_ROWS_FIELD_EQUALS	Every matched row equals the expected value
NO_ROW_OLDER_THAN_DAYS	No matched row is older than the tolerance

Filters accept the tokens "@today" and ["<", "@days_ago:90"], so a freshness check does not need editing every day.

Half of a compliance programme's evidence is already sitting in the ERP: leavers still holding roles, dormant privileged accounts, vendors processing personal data with no signed DPA, policies past review. Writing a Python handler for each is disproportionate work for what is really a query plus a threshold.

Two guards worth knowing about

Field names in filters and conditions are validated against the target DocType's own metadata at save time rather than at run time. A typo fails in front of the person writing it, not at two in the morning inside a scheduled run where it would surface as an unexplained collection fault.

Fields of type Password, and fieldnames shaped like credentials — `api_secret`, `token`, `private_key` — are refused outright. Evidence artifacts are hashed, attached and retained under a retention policy, and a credential must never become a permanent audit record.

Each failing row becomes a resource on the emitted finding, so the output is a task list rather than a status. "Twelve leavers still hold roles" names the twelve.

Adapted from `alphax_grc`, with the evaluated string removed

The GRC Evidence Rule DocType in `alphax_grc` stores its pass logic as free text — "IF `match.group(1) == match.group(2)` THEN PASS" — and evaluates it in Python. That turns anyone who can create a rule record into a holder of arbitrary code execution on the site. Here the condition is an enumerated Select and the engine dispatches on it.

5.7 Validation rules

A rule states what the collected output must show. Four rule types are available.

Rule type	Use
Regex Capture	Extract one or more groups from text output and compare them
JSON Path	Read a dotted path from JSON output and compare the value
JSON Key Equals	Assert a key holds an expected literal value
Non-Empty Output	Assert the fetcher returned anything at all

The logic applied to the extracted value is an enumerated Select — for example `ALL_GROUPS_EQUAL`, `GROUP1_EQUALS_GROUP2`, `VALUE_GTE_EXPECTED`, `VALUE_LTE_EXPECTED`, `MATCH_ABSENT`. The engine dispatches on that enum.

Why the logic is enumerated rather than free text

The upstream Paramify catalogue stores rule logic as strings such as "IF `match.group(1) == match.group(2)` THEN PASS" and evaluates them in Python. Inside a Frappe site that turns any user who can create a Fetcher record into a holder of arbitrary code execution. The expressiveness given up here is small; the attack surface removed is not.

Worst-wins across rules

A fetcher may carry several rules. The outcome is the worst across all of them: any fail makes the fetcher fail, and inconclusive outranks pass. The recorded severity is the highest severity among the failing rules.

The absent-rule case

A fetcher with no validation rules records **inconclusive**, never **pass**. Collecting a file is not the same as demonstrating a control, and a passing status that means nothing more than "the script ran" will eventually be relied upon by someone who does not know that.

5.8 Shell fetcher sandboxing

Shell handlers execute with `shell=False` against an `argv` list. The script path is resolved with `os.path.realpath` inside the connector's declared `script_root`. Absolute paths, parent-directory traversal, and symlinks pointing outside the root are all rejected before execution.

The reasoning is that a Fetcher record is operator-supplied data. Desk access to create one must not become a remote code execution primitive against the host. Shell connectors therefore ship disabled with no script root configured, so the path has to be set deliberately by someone who understands what they are pointing at.

6 The finding contract

Every finding, whether produced internally or posted by an external connector, must satisfy contract version 1.0.0. One finding document describes one resource and carries one or more control evaluations.

```
{
  "schema_version": "1.0.0",
  "source": "aws-inspector",
  "source_version": "2026.04.01",
  "run_id": "01HXKJ...",
  "collected_at": "2026-04-13T15:04:05Z",
  "resource": {
    "type": "aws_s3_bucket",
    "id": "acme-prod-logs",
    "arn": "arn:aws:s3:::acme-prod-logs",
    "region": "us-east-1"
  },
  "evaluations": [
    {
      "control_framework": "NCC",
      "control_id": "CRY-01",
      "status": "fail",
      "severity": "high",
      "message": "Bucket has no default encryption configured",
      "remediation": {
        "summary": "Enable SSE-KMS with a customer-managed key",
        "effort_hours": 1,
        "automation": "auto_fixable"
      }
    }
  ]
}
```

6.1 Status and severity are independent

Field	Values	Meaning
status	pass, fail, not_applicable, inconclusive, skipped	What the check found
severity	critical, high, medium, low, info	How much it matters if the control is failing
automation	auto_fixable, semi_automated, manual, design_change	What remediation would take

Conflating the two is a common design error. A failing low-severity control is real and belongs in the register; it is simply not urgent. Keeping the fields separate is what lets the remediation plan sort by impact rather than by alarm.

The **inconclusive** status carries specific meaning: the tool tried and could not determine the answer. A dropped API call, a permission the collection role lacks, a malformed response. It is never scored as a pass, and a control whose only evidence is inconclusive is reported as assessed-but-undetermined rather than compliant.

6.2 Contract rules enforced at ingest

- A fail evaluation must carry both a message and a severity. An unexplained failure is not actionable.
- An inconclusive evaluation must carry a message explaining why the determination could not be made.
- A pass evaluation needs neither, though both are accepted.
- At least one evaluation is required. A finding with no evaluations describes nothing.
- collected_at must be a valid ISO-8601 timestamp; offsets are normalised to UTC on write.
- effort_hours, where present, must be non-negative.
- The schema major version must match. A 2.x document is rejected rather than best-guessed.

6.3 Ingest API

```
curl -X POST https://yoursite/api/method/ \
  neogrc.api.ingest_findings \
  -H "Authorization: token <api_key>:<api_secret>" \
  -H "Content-Type: application/json" \
  -d '{"findings": [ ... ], "dry_run": 1}'
```

Behaviour	Detail
Whole-batch validation	The entire batch is validated before anything is written. A partially ingested run is worse than a rejected one, because a gap assessment over half a connector's output reports the missing half as uncovered and nobody notices.
Idempotent	Keyed on run_id, source, resource_id and resource_type. Re-posting the same batch updates in place rather than duplicating.
Dry run	dry_run: 1 validates and reports errors without writing. Use it when building a new connector.
Batch ceiling	2,000 findings per call. Larger collections should page.
Redaction	Configured key names — password, token, secret, iqama, national_id and others — are stripped from raw resource attributes on write.
Role-guarded	Requires NeoGRC Manager or NeoGRC Engineer. Errors are indexed, so findings[7] tells the connector author exactly which document failed.

7 Crosswalk and gap assessment

7.1 Two-hop resolution

every mapping pivots through the canonical framework — two hops, never three

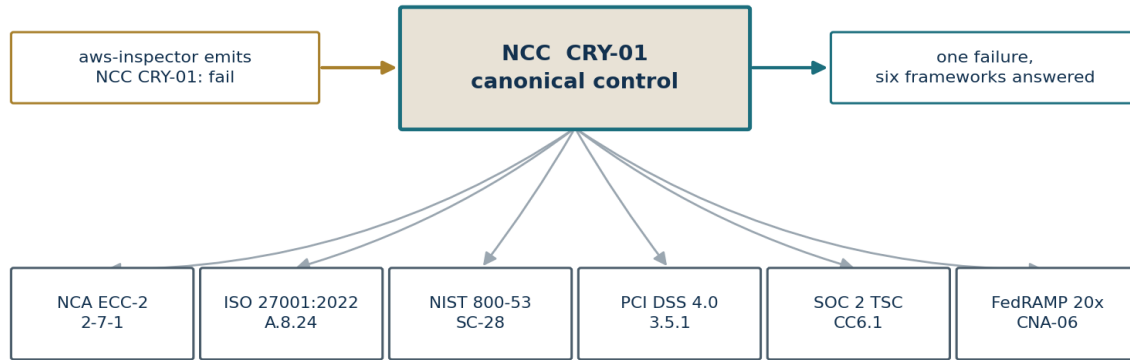


Figure 4 — One canonical failure resolving into six framework obligations

Mappings resolve through the canonical framework and no further: source control to canonical, canonical to target. Transitive chaining beyond two hops is refused, because a mapping three steps removed from the evidence is a guess dressed as a control assertion.

The canonical framework is configurable in GRC Engine Settings and ships as NCC — Neotec Canonical Controls, 62 controls across 16 families. Organisations licensing the Secure Controls Framework can import it and designate it canonical instead; nothing else in the application changes.

Conflicting requirements

The Conflicting Requirements action on any control surfaces mapped controls whose severity or automation posture disagree across frameworks, and names which obligation binds. Where two frameworks disagree, the stricter is the one the organisation actually has to meet — the tool makes that explicit rather than averaging it away.

7.2 Running an assessment

1. Create a GRC Gap Assessment and name the period it covers.
2. Add one or more frameworks to the scope table. Multi-framework assessments are the normal case.
3. Optionally restrict the source connectors considered.
4. Click Run Assessment. Execution is queued.
5. Review the results table and the scorecard, then submit to freeze the assessment as an audit record.

Submission matters beyond workflow tidiness: the retention purge will not delete evidence artifacts that sit behind a submitted assessment. Submitting is what makes the evidence permanent.

7.3 How controls are scored

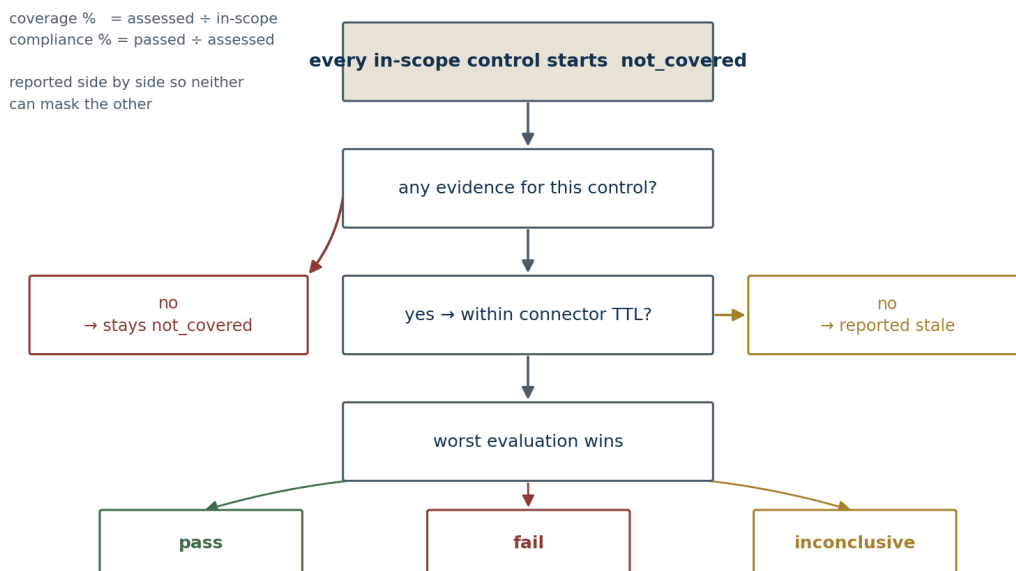


Figure 5 — Scoring path for a single control, and the two percentages reported

Every in-scope control begins as `not_covered`. Evidence promotes it. That default is the single most important decision in the scoring model.

The common failure of automated compliance tooling is to report absence of evidence as absence of a problem — a control nothing tested comes back clean, and the dashboard is greener than the organisation. Starting at `not_covered` inverts that. A control with no evidence is a gap until something demonstrates otherwise.

Worst-wins within a control

One failing evaluation fails the control, however many resources passed. Ninety-nine encrypted buckets and one unencrypted bucket is a failing encryption control, because the unencrypted one is the one that will appear in the incident report.

Two percentages, reported separately

Metric	Formula	Answers
Automated coverage	assessed ÷ in-scope	How much of the framework did we actually test?
Compliance	passed ÷ assessed	Of what we tested, how much passed?

Neither can mask the other. A single blended figure would let a narrow, clean scope produce a reassuring number: 100% compliance across a 4% coverage means almost nothing was examined. Both figures appear on the scorecard together, and the second is meaningless without the first.

Staleness

Findings older than their originating connector's cache TTL are excluded from scoring and reported as stale, unless Include Stale Findings is ticked on the assessment. Evidence has a shelf life; a passing result from eight months ago describes a system that may no longer exist.

7.4 Multi-framework optimisation

The Multi Framework Optimisation report ranks canonical controls by return on remediation effort — weighted risk closed per hour of work, accounting for how many in-scope frameworks each control satisfies.

Its practical use is sequencing. A control mapping to six frameworks at four hours of effort outranks one mapping to a single framework at the same cost, and the report says so with a number rather than leaving it to whoever is loudest in the remediation meeting.

8 Operations

8.1 Scheduled jobs

Frequency	Job	Behaviour
Hourly	Queue due evidence sets	Skips any set with a run still in flight, so a slow collection never stacks
Daily	Expire exceptions	Exceptions past their expiry date are closed automatically
Daily	Flag overdue reviews	Owners notified of overdue risk, policy and vendor reviews
Daily	Snapshot metrics	KPI and KRI values captured for trend reporting
Daily	Purge expired artifacts	Retention enforced, protecting evidence behind submitted assessments

8.2 Reports

Report	Answers
Control Coverage Matrix	Which controls can be evidenced automatically, and by which fetcher
Findings by Control	Open failures grouped by control, severity-ordered
Evidence Freshness	Last successful collection per fetcher against its TTL
Multi Framework Optimisation	Which remediations close the most weighted risk per hour

8.3 Data residency

Residency Mode in GRC Engine Settings defaults to On-Premise Only. In that mode the controller rejects any AI endpoint that is not local unless it has been explicitly allow-listed, which is enforced at validation time rather than left to operator discipline.

Narrative generation is disabled by default. When enabled it targets a self-hosted Ollama endpoint. There is no hosted-LLM code path in the application, which is a deliberate constraint for KSA PDPL deployments where evidence content — user lists, access reviews, infrastructure inventory — must not leave the Kingdom.

8.4 Troubleshooting

Symptom	Likely cause
Evidence set never runs on schedule	Scheduler disabled, or the set itself is disabled. Check bench doctor.
Run halts immediately at exit 5	Preflight failed. The run record names the missing binary or environment variable.
All fetchers return inconclusive	No validation rules defined on those fetchers. Expected behaviour, not a

Symptom	Likely cause
	fault.
Coverage is 0% after a successful run	Fetchers have no control links, so findings resolve to no control in scope.
Ingest returns a validation error list	Errors are indexed by position. findings[7] identifies the failing document.
Crosswalk shows no targets	No canonical framework designated, or no crosswalk edges for that control.
Assessment ignores recent findings	Findings are past connector TTL. Tick Include Stale Findings, or shorten the cadence.

9 Licensing and attribution

No normative control text from any licensed standard ships with this application

Control identifiers and clause numbers are referenced, which is how every GRC product cross-references. Titles, objectives and test procedures for licensed frameworks are not reproduced.

9.1 Framework status

Framework	Controls	Status
NCC (canonical)	62	Original wording. Freely editable and redistributable.
NCA ECC-2:2024	193	Real control codes with domain and subdomain structure, 90 subcontrols. Identifiers only.
NIST CSF 2.0	106	Subcategory codes and titles. US Government work, public domain.
NIST SP 800-53 r5	58	Identifiers. Public domain.
EU GDPR	43	Article numbers and titles. Published legislation.
ISO 27001:2022 · ISO 42001:2023 · ISO 22301:2019	114	Identifiers only. Normative text is ISO/IEC property.
KSA PDPL	31	Articles, Implementing Regulations, Cross-Border rules and NDMO guidance.
PCI DSS 4.0 · CIS v8 · SOC 2 TSC	62	Identifiers only. Text remains with the respective bodies.
FedRAMP 20x KSI · NCA CCC · SAMA CSF	33	Key indicators and control identifiers.

702 controls across 15 frameworks, joined by 1,121 crosswalk edges.

What was deliberately left out

The NCA ECC-2 catalogue used as the structural source for the 193 control codes also carries the verbatim normative Arabic text from the official GECC-2024 spreadsheet, along with the implementation-method and expected-output columns. None of that is reproduced here. What was taken is the control codes, the domain and subdomain labels, and the subcontrol hierarchy.

The same source's English column was machine-damaged in any case — 145 of its 198 rows are empty, and every one of the remaining 53 begins mid-sentence or carries footnote artefacts — so it would not have been usable even had the licensing allowed it.

Binding and related mappings

Every crosswalk edge carries a relationship. Only equivalent, subset and superset carry coverage into a gap assessment. Related edges appear in the crosswalk viewer, marked as not carrying coverage, and are excluded from scoring.

This matters because 910 of the 1,121 shipped edges are derived at family level — canonical family to framework subdomain — rather than control to control. Treating those as binding would let a single passing governance check mark dozens of target controls as covered, which is exactly the coverage inflation described in section 7.3. Narrow them to control level as you validate each one, and change the relationship to equivalent as you go.

Expect low ECC coverage on the first assessment

The 356 NCA ECC-2 edges shipped are family-level derivations. They will not score until a consultant narrows them to control level. That is the correct behaviour: a mapping nobody has checked should not answer a regulator's control.

9.2 Loading licensed text

Where the organisation holds a licence, replace the stub titles and objectives in place. The controls, crosswalk edges and any findings already mapped to them are unaffected — only the descriptive fields change. This is the intended workflow rather than a workaround.

9.3 The build-time guard

verify_tree.py fails the build if standard-clause phrasing appears anywhere in the seed data. It is a blunt check on characteristic phrasing, and it is deliberately blunt: it caught its own author's draft ISO disclaimer during development, which is the check doing its job.

9.4 Reference projects

Three open-source projects informed the design. No code was copied from any of them; what was taken is the shape of the data contract and the collection pipeline.

Project	What it informed
claude-grc-engineering (NeoGRC Engineering Club)	The finding schema v1.0.0 data contract, the risk, exception, vendor, policy and metric record shapes, the canonical-control crosswalk model, and the connector quality bar
evidence-fetchers (Paramify)	Fetcher catalogue structure, evidence sets mapped to FedRAMP 20x Key Security Indicators, regex validation rules, timestamped run directories, and structured exit codes
NeoGRC Engineering Club directory	Specialisation and framework controlled vocabulary

This application is not affiliated with, or endorsed by, any of the above.

10 Extending the application

10.1 Adding a connector

1. Create a GRC Connector: identifier, handler type, auth mode, required binaries and environment variables, cache TTL.
2. For shell handlers, set `script_root` to the directory holding the collection scripts. It is not seeded and must be set deliberately.
3. Create a GRC Evidence Fetcher for each check, with its script or handler path.
4. Map each fetcher to canonical controls in its control link table.
5. Add validation rules. Without them the fetcher records inconclusive.
6. Run Preflight Check from the connector, then run one fetcher in isolation.
7. Add the fetcher to an evidence set and set a schedule.

10.2 Adding a framework

1. Create a GRC Framework record with its code, version and attribution note.
2. Load its controls, at minimum identifiers and titles.
3. Create crosswalk edges from canonical controls to the new framework's controls.
4. Add the framework to a gap assessment scope and run it.

No connector is modified and no evidence is re-collected. Existing findings begin answering the new framework as soon as the crosswalk edges exist — this is the payoff for the canonical pivot described in section 2.1.

10.3 Development checks

```
python3 verify_tree.py          # structural + licensing guard
python3 tests/test_offline_logic.py # contract + rule engine, no bench
bench --site yoursite run-tests --app neogrc
```

`verify_tree.py` checks field-order consistency against declared fields, link and child-table targets, patch placement and resolution, hooks references, permission guards on every whitelisted method, seed-data referential integrity, and the absence of committed Server Scripts.

`test_offline_logic.py` runs the contract validator and rule engine against a minimal Frappe stub — 34 assertions, no bench and no database. These two modules decide whether a compliance record is auditable, so they are worth being able to test in a pipeline that has no site.

10.4 Suggested next steps

Item	Why
Narrow the derived ECC and CSF mappings	910 edges are family-level and do not carry coverage until checked at control level

Item	Why
Load licensed NCA ECC-2 control text	The 193 ECC codes and structure are real, but titles and objectives need your licensed copy
Enable and credential one cloud connector	Internal evidence alone cannot lift coverage past the site boundary
Populate the risk register	Findings gain business context when they link to a scored risk
Set review cadences on policies and vendors	Three internal fetchers evidence GRC hygiene and need cadence data to assess against
Schedule the quarterly cloud set	Quarterly cadence suits posture evidence that changes slowly but must be shown as current

Neotec Integrated Solutions · Riyadh, Kingdom of Saudi Arabia · support@neotec.ai

NeoGRC v0.2.0 — Implementor Manual — 12 August 2026